

PIXEL SOFTWARE DESIGN

Département de Recherche & Développement

Intelligence Artificielle Avancée

& Federated Learning

- Fondations de l'intelligence artificielle
- Algorithmes d'apprentissage fédéré
- Confidentialité, attaques et défenses
- Applications et grands modèles fédérés
- Framework FedX de référence
- Nouveaux fronts de recherche
- Travaux pratiques, projets et évaluation

Auteur	**Ghiloufi Jamel**
Fonction	**Architecte Logiciel Principal**
Direction	**Département de Recherche & Développement**
Service	**R&D - Intelligence Artificielle Avancée & Apprentissage Fédéré**
Niveau	**Recherche**
Édition	**Édition 2026 – 1.0**
Copyright	**© Pixel Software Design**

Ce document est la propriété exclusive de Pixel Software Design. Toute reproduction, représentation, adaptation ou diffusion, totale ou partielle, par quelque procédé que ce soit et sur quelque support que ce soit, est interdite sans autorisation écrite préalable de l'auteur.

Copyright et mentions légales

Éditeur	Pixel Software Design
Auteur	Ghiloufi Jamel
Fonction	Architecte Logiciel Principal
Département	Département de Recherche & Développement
Intitulé	Intelligence Artificielle Avancée & Federated Learning
Sous-titre	Fondations mathématiques de l'apprentissage automatique, algorithmes fédérés, protection de la vie privée, défense contre les attaques, grands modèles de langage fédérés, systèmes embarqués, renforcement appris, applications critiques (santé, V2X), plateforme de référence FedX et feuille de route de recherche.
Édition	Édition 2026 (version 1.0)
Langue	Français
Niveau	Recherche
Collection	Supports de formation et de recherche, Pixel Software Design

Droits d'auteur

© 2026 Pixel Software Design. Tous droits réservés.

Ce document est la propriété exclusive de Pixel Software Design. Toute reproduction, représentation, adaptation ou diffusion, totale ou partielle, par quelque procédé que ce soit et sur quelque support que ce soit, est interdite sans autorisation écrite préalable de l'auteur.

Droits de l'auteur. Ghiloufi Jamel est l'auteur unique de l'ensemble du contenu scientifique, pédagogique et technique de cet ouvrage. Toute adaptation, traduction, reproduction ou exploitation, y compris partielle, requiert une autorisation écrite préalable.

Droits des tiers. Les algorithmes, modèles et travaux cités restent la propriété de leurs auteurs respectifs. Les références employées figurent en fin d'ouvrage, ce qui permet de remonter au travail original pour toute affirmation chiffrée.

Marques et licences externes. PyTorch, TensorFlow, Flower, Opacus, MONAI, scikit-learn et PySyft sont des logiciels libres sous licences respectives (BSD, Apache 2.0, MIT). Leur usage dans ce document est purement pédagogique et n'emporte aucune prescription commerciale.

Mentions relatives aux données et aux expérimentations

Document pédagogique à usage de formation interne. Les exemples de code fournis sont livrés à titre pédagogique ; les résultats numériques illustrés sont issus d'expérimentations conduites dans un cadre de recherche et ne constituent pas des performances de production.

Ce support synthétise les travaux de recherche du Département de Recherche & Développement de Pixel Software Design sur l'intelligence artificielle distribuée, l'apprentissage fédéré, la protection de la vie privée et les systèmes embarqués intelligents.

Sommaire

Structure détaillée du cours

Partie 1 — Intelligence artificielle de A à Z 8

1.1 Fondations et logique symbolique 9

1.2 Apprentissage automatique et profond 11

1.3 IA générative et modèles de langage 14

1.4 MLOps, éthique et responsabilité 15

Annexes 20

A.1 Liste des abréviations 21

A.2 Table de notation 21

Partie	Thème	Contenu	Compétence visée	Durée
Partie 1	Fondations	Statistique, optimisation, courbes d'apprentissage	Comprendre et formaliser	2-3 semaines
Partie 2	Algorithmes FL	FedAvg, convergence, hétérogénéité, système	Implémenter et régler	3-4 semaines
Partie 3	Confidentialité	DP, attaques, défenses, agrégation sécurisée	Auditer et sécuriser	2-3 semaines
Partie 4	Applications	FedLLM, santé, V2X, FedRL, TinyML, graphes	Transposer à un domaine	3-4 semaines
Partie 5	Framework FedX	Architecture, API, benchmarks, exploitation	Industrialiser	2 semaines
Partie 6	Nouveaux fronts	FedCL, FedCI, SNN, FedGL, feuille de route	Ouvrir la recherche	2 semaines
Partie 7	TP et projets	9 labs, 5 projets, corrigés, évaluation	Faire et rendre	4-6 semaines

Préface

Comment utiliser ce cours

Ce support est conçu pour un parcours de formation et de recherche couvrant l'ensemble de la chaîne de valeur de l'intelligence artificielle distribuée : les fondations mathématiques, les algorithmes d'apprentissage fédéré, les menaces et défenses, les applications de pointe, une plateforme de référence opérationnelle, et enfin les nouveaux fronts ouverts par la recherche du Département.

Ce n'est pas un manuel de référence à lire d'un trait : c'est un **manuel opératoire**. Chaque chapitre alterne exposition, formalisation, mise en œuvre numérique et évaluation. Les listings sont complets et exécutables ; les tableaux comparatifs servent de grille de décision ; les schémas d'architecture servent de support de revue de conception.

Public et prérequis

Le contenu s'adresse à des étudiants de master et de doctorat, à des ingénieurs IA et à des praticiens de la recherche appliquée. Les prérequis sont la connaissance de l'apprentissage automatique classique, de l'algorithmique numérique et des bases de probabilités. Les parties 1 à 3 sont auto-suffisantes pour un étudiant qui découvre le domaine ; les parties 4 à 7 supposent la mécanique et les outils présentés dans la partie 2.

Conventions de lecture. Une expression mathématique est encadrée par deux signes dollars ; un identifiant, un fichier ou une commande est écrit en caractères à chasse fixe ; les encadrés *Définition*, *Théorème*, *Méthode* et *Attention* signalent les points à maîtriser et les pièges fréquents. Les renvois de la forme « voir chapitre 3.2 » désignent le second chapitre de la troisième partie.

Progression pédagogique

Des fondations symboliques à l'industrialisation

Partie 1

Intelligence artificielle de A à Z

Un cours panoramique qui traversé toute la discipline dans l'ordre des dépendances : on représente d'abord la connaissance, on apprend ensuite à l'exploiter, puis on la genere, enfin on l'industrialise. Chaque module reprend les notions du précédent et indique ce qu'il faut savoir faire, pas seulement comprendre.

Objectifs de la partie

- Représenter une connaissance sous forme exploitable : règles, prédicats, ontologies.
- Explorer un espace d'états de façon exacte (A*) ou sous contrainte de temps (minimax).
- Choisir entre modèle linéaire, forêt, réseau profond et modèle de langage.
- Comprendre l'auto-attention, l'encodage positionnel et l'ajustement de faible rang.
- Mettre en place un cycle MLOps, une API d'inférence et une surveillance de production.
- Auditer un modèle sur ses biais, son explicabilité et la gouvernance de ses données.

Plan de la partie

1. Fondations et logique symbolique
2. Apprentissage automatique et profond
3. IA générative et modèles de langage
4. MLOps, éthique et responsabilité

Chapitre 1.1

Fondations et logique symbolique

Tout le reste du cours repose sur une idée : rendre la connaissance calculable.

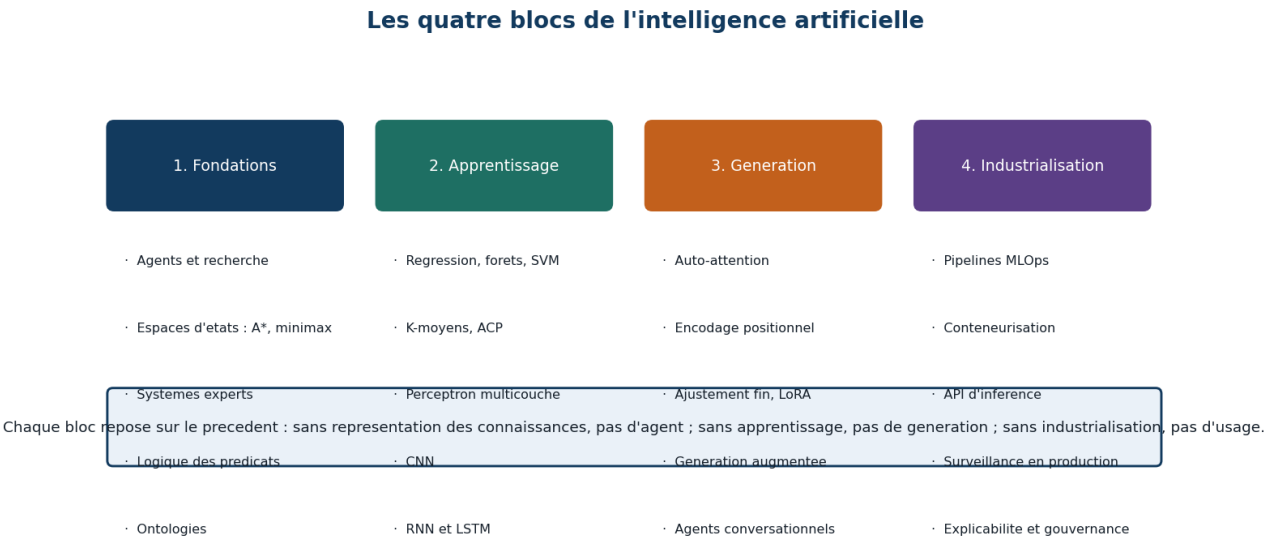


Figure 1 — Les quatre blocs de l'intelligence artificielle

L'agent rationnel

L'agent rationnel est le modèle de référence de toute intelligence artificielle : il perçoit son environnement, choisit une action parmi celles possibles, et apprend de la conséquence obtenue. Ce schéma en quatre temps suffit à englober la recherche sur les agents, la planification automatique et l'apprentissage par renforcement.

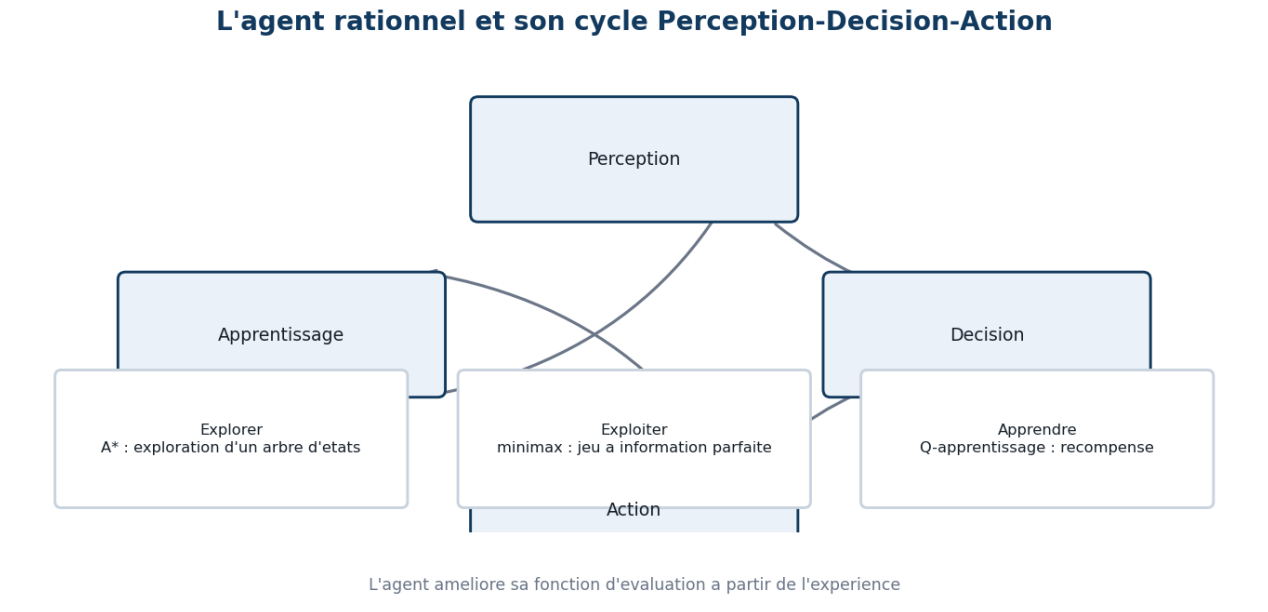


Figure 2 — Le cycle perception-décision-action de l'agent

Stratégie	Principe	Optimalité	Usage
Recherche en largeur	on explore tous les successeurscomplète, mais coûteuse	espaces profonds, mémoire suffisante	
Recherche gloutonne	on ne garde que le meilleur successeur	incomplète, rapide	problèmes bien structurés
A*	gloutonne guidée par une estimation de coût restant	optimale si l'estimation est admissible	planification, robotique, jeux
Minimax	alternance maximisation et minimisation	optimale contre un adversaire parfait	jeux à deux joueurs
Apprentissage par renforcement	approximation apprise	approximative, amortie	systèmes trop vastes pour l'énumérer

Tableau 1 – Trois stratégies d'exploration d'un espace d'états

A* et minimax partagent la même question : quel ordre d'exploration rend la garantie atteignable dans le budget disponible ?

$$f^{*}(n) = \min_{a \in A(n)} \left[c(n, a) + h(n') \right]$$
$$V(X) = \max_{a} \left[r(X, a) + \gamma E\left[V(X')\right] \right] \quad \text{(Z.1)}$$

La fonction d'évaluation d'A* est une estimation du coût restant ; l'équation de Bellman est la condition d'équilibre d'une politique. Les deux expressions disent la même chose : valeur estimée plus valeur future.

> **Astuce**
> Une heuristique admissible (qui ne surestime jamais le coût restant) est la condition exacte pour que la recherche A* soit optimale. Une heuristique qui surestime rend la recherche plus rapide mais produit des solutions dégradées — arbitrage direct entre garantie et performance.

Représenter la connaissance : systèmes experts et règles

Un système expert combine une base de règles et un moteur d'inférence. Le nombre d'expert humain se transforme en règles du type « si <prémisse> alors <conclusion> », et le moteur enchaîne les règles pour répondre à une question posée.

- **Avantage :** explicabilité totale. On peut nommer la règle qui a produit la conclusion.
- **Avantage :** extrapolation contrôlée. Le système ne dit rien là où aucune règle ne s'applique.
- **Limite :** l'acquisition des règles est la partie la plus coûteuse et la moins automatisable du projet.
- **Limite :** la maintenance explose à mesure que le domaine s'étend — c'est la cause principale du premier hiver de l'IA.

Logique des prédicats et ontologies

La logique des prédicats étend la logique propositionnelle par des variables, des prédicats et des quantificateurs. Elle permet d'exprimer une règle générale et de l'appliquer par unification.

$$\forall x \left[\text{Homme}(x) \rightarrow \text{Mortel}(x) \right]$$
$$p(X) \wedge \forall Y\ p(Y) \Rightarrow \exists Z\ q(X, Z) \quad \text{(Z.2)}$$

La première formule est un syllogisme : tout homme est mortel. La seconde introduit la quantification existentielle, c'est-à-dire la capacité d'inventer une constante qui n'était pas dans les prémisses – c'est la source de la puissance comme de l'explosion combinatoire du raisonnement.

	Logique des prédicats	Ontologie
Élément	Formule et prédicat	Classe, individu, relation
Rôle	raisonner, déduire	décrire,typer, relier
Exemple	tout homme est mortel	un patient a une pathologie
Contradiction gérée	logiquement, par démonstration	par incohérence de l'ontologie
Outil	moteur de déduction, Prolog	influenceur, éditeur ontologique
Limite	explosion combinatoire	coût de construction et de maintenance

Tableau 2 – Logique des prédicats et ontologies : deux usages distincts

Les deux sont complémentaires : l'ontologie décrit le monde, la logique en tire les conséquences.

Inverser un quantificateur. « Toute femme est médecin » ne signifie pas « toute médecin est femme ». Les moteurs de déduction ne font pas cette erreur ; les guides humains, si. Et, un modèle génératif produit aussi des généralisations de ce type sans le moindre signal dans les données.

L'erreur de raisonnement la plus coûteuse

A retenir

- L'agent rationnel est le modèle de référence de toute la discipline.
- A* et minimax répondent à la même question : quel ordre d'exploration rend la garantie atteignable ?
- La logique des prédicats rend la connaissance calculable mais souffre d'explosion combinatoire.
- L'ontologie décrit le monde, la logique en tire les conséquences : les deux sont complémentaires.

Chapitre 1.2

Apprentissage automatique et profond

Quand l'écriture des règles devient impossible, on apend à les déduire des exemples.

L'apprentissage automatique remplace l'écriture explicite des règles par leur extraction à partir d'exemples. Le changement de paradigme est radical : on passe d'un système qui sait pourquoi il répond à un système qui sait seulement quoi répondre.

Apprentissage supervisé classique

Algorithme	Principe	Données adaptées	Limite principale
Régression linéaire	ajuste une droitetabulaires, relation linéaire	sensible aux valeurs extrêmes	
Régression logistique	ajuste une probabilité	tabulaires, classification binaire	séparation linéaire seulement
Machines à vecteurs de support	sépare au plus près possible	petites dimensions, frontières nettes	coût en forte dimension
Arbre de décision	pose des questions successives	tabulaires, explicabilité	instable, n'extrapole pas
Forêt aléatoire	moyenne de nombreux arbres	tabulaires, bon rapport gain/complexité	modèle volumineux
Renforcement de gradient	corrige les erreurs d'un modèle faible	tabulaires, meilleur score	sensible aux hyperparamètres

Tableau 1 – Les algorithmes supervisés et ce qu'ils attendent des données

Sur données tabulaires, le renforcement de gradient est presque toujours le meilleur premier essai ; sur texte ou image, un réseau profond le dépasse très largement.

Apprentissage non supervisé

Sans étiquette, l'objectif change : on cherche une structure. Les deux familles les plus utilisées sont le regroupement, qui cherche des groupes, et la réduction de dimension, qui compresse en conservant l'essentiel.

- **K-moyens :** partition en K groupes par minimisation de la dispersion interne. Sensible à l'initialisation et à l'échelle des variables — d'où la nécessité de normaliser.
- **ACP :** projette les données sur les axes de plus forte variance. Interprétable, mais linéaire : il ne voit pas les structures courbes.
- **Réduction non linéaire :** projection sur des vecteurs, méthodes de voisinage. Bien plus expressive, bien moins lisible.

Choisir son première méthode, en quatre questions

- **La donnée est-elle étiquetée ?** Si non, on part de K-moyens.

2. **La donnée est-elle tabulaire ou structurée ?** Si oui : forêt aléatoire comme référence, puis renforcement de gradient.
3. **La donnée est-elle une image, un son, du texte ou une série ?** Si oui : réseau convolutif, récurrent ou transformateur.
4. **Quelle erreur coûte le plus cher ?** Le choix de la métrique et de la classe de référence en découle.

Réseaux profonds

Un réseau profond est une composition de transformations non linéaires. Trois familles de couches correspondent à trois hypothèses sur la structure des données.

Réseau	Hypothèse	Structure	Domaine
Perceptron multicouche	aucune structure particulièrecouches denses fully connectées	vecteurs, tabulaire	
Réseau convolutif	grille régulière, voisinage local	filtres glissants partagés	image, signal, audio
Réseau récurrent (RNN, LSTM)	dépendance strictement ordonnée	état caché propagé dans le temps	séries temporelles, audio, texte
Transformateur	dépendance entre positions quelconques	auto-attention multi-têtes	texte, code, multimodal

Tableau 2 – Perceptron, convolution, récurrence : trois hypothèses

- **Perceptron multicouche :** la brique de base. Sans structure spatiale ou temporelle, mais suffisante pour des vecteurs.
- **LSTM :** la cellule à portes corrige le problème de l'oubli des réseaux récurrents simples en apprenant quels éléments garder.
- **CNN :** les filtres partagés rendent le modèle invariant par translation et réduit fortement le nombre de paramètres.
- **Transformateur :** abandonné l'ordre au profit d'une dépendance globale apprise, au prix d'un coût quadratique.

```python
import torch
import torch.nn as nn


class BlocResiduel(nn.Module):
"""Bloc de reference : convolution, normalisation, activation, saut.

L'ordre normalisation-activation-convolution n'est pas indifferent :
la normalisation doit preceder l'activation pour stabiliser
l'entrainement, et la connexion residuelle doit envelopper l'ensemble
du bloc pour que le gradient circule sans attenuation.
"""

def __init__(self, entrees: int, sorties: int, facteur: int = 4):
super().__init__()
reducteur = entrees // facteur
self.norm1 = nn.GroupNorm(8, entrees)
self.conv1 = nn.Conv2d(entrees, reducteur, kernel_size=3, padding=1)
self.norm2 = nn.GroupNorm(8, reducteur)
self.conv2 = nn.Conv2d(reducteur, reducteur, kernel_size=3, padding=1)
self.norm3 = nn.GroupNorm(8, reducteur)
self.conv3 = nn.Conv2d(reducteur, sorties, kernel_size=3, padding=1)
self.saut = (nn.Identity() if entrees == sorties
else nn.Conv2d(entrees, sorties, kernel_size=1))
self.activation = nn.GELU()

def forward(self, x: torch.Tensor) -> torch.Tensor:
y = self.conv1(self.activation(self.norm1(x)))
y = self.conv2(self.activation(self.norm2(y)))
y = self.conv3(self.activation(self.norm3(y)))
return self.saut(x) + y


class Extracteur(nn.Module):
"""Encodeur convolutif : convolution d'entree, blocs residuels, moyenne."""

def __init__(self, canaux: int = 3, largeur: int = 64, profondeur: int = 4,
normative: bool = True):
super().__init__()
self.normative = normative
self.entree = nn.Conv2d(canaux, largeur, kernel_size=5, padding=2)
self.blocs = nn.ModuleList([
BlocResiduel(largeur if i == 0 else largeur * 4,
largeur * 4 if i < profondeur - 1 else largeur)
for i in range(profondeur)])
self.normale = nn.LayerNorm(largeur)

def forward(self, x: torch.Tensor) -> torch.Tensor:
if self.normative:
# centrage par image : la moyenne est calculee par lot et par canal
x = x - x.mean(dim=(0, 2, 3), keepdim=True)
y = self.entree(x)
for bloc in self.blocs:
y = bloc(y)
y = self.normale(y.mean(dim=(2, 3)))
return y
```

Listing 1 — Encodeur convolutif avec blocs résiduels et normalisation

A retenir

- Le changement de paradigme est radical : de règles explicites à l'extraction de règles.
- Sur données tabulaires, le renforcement de gradient est le meilleur premier essai.
- Réduire de dimension et regrouper exigent une normalisation préalable.
- Chaque famille de couche encode une hypothèse sur la structure des données.

Chapitre 1.3

IA générative et modèles de langage

Un modèle de langage est un modèle qui prédit la suite — et toute la génération en découle.

Auto-attention : le mécanisme central

L'auto-attention permet à chaque position de comparer son représentation à toutes les autres, et de pondérer ces liens par apprentissage. Elle remplace la récurrence, ce qui rend l'entraînement entièrement parallèle.

$$\text{Attention}(Q,K,V) = \text{softmax}\leq ft((Q\,K^{\top})/(\sqrt{(d_k)}) + M)\,V$$
$$\text{Entête}_i = \text{Attention}\leq ft(X\,W_i^{Q}, X\,W_i^{K}, X\,W_i^{V}) \quad \text{(Z.3)}$$

Trois projections, trois rôles : la requête identifie ce que la position cherche, la clé indique ce qu'elle offre, et la valeur porte le contenu. L'échelle $1/\sqrt{(d_k)}$ est indispensable : sans elle la softmax sature et le modèle n'apprend rien.

Pourquoi l'encodage positionnel est nécessaire

L'auto-attention est permutationnelle : elle ne sait pas, seule, dans quel ordre les jetons arrivent. L'encodage positionnel réinjecte cette information en ajoutant à chaque représentation une signature de sa position.

Famille	Principe	Atout	Limite
Absolu	une table apprise pour chaque positionsimple, précis en position	limité à la longueur apprise	
Relatif	décalage apprise entre positions	extensibilité en longueur	position approximative
Rotationnel	rotation de fréquence par dimension	propriété relative naturelle	qualité variable selon la règle
Fondu	signal slowly-growing noyé dans le mot	s'intègre à l'architecture	dégradation si la longueur dépassela fenêtre d'entraînement

Tableau 1 – Trois familles d'encodage positionnel

Ajustement : du préentraînement à l'usage

Un modèle de langage se préentraîne sur un corpus entier, puis s'ajuste sur une tâche ou un domaine. L'ajustement complet est coûteux et détruit le bénéfice du préentraînement ; l'ajustement de faible rang corrige le problème.

$$W_{t+1} = W_t + (\alpha)/(r)\,B_t\,A_t,\; A_t \in \mathbb{R}^{r\times d},\; B_t \in \mathbb{R}^{d\times r},\; r \ll d \quad \text{(Z.4)}$$

Seuls A et B sont entraînés, soit une fraction de la matrice W. Le facteur α/r compense la réduction de rang : l'oublier rend l'adaptation instable d'une configuration à l'autre.

Voie	Paramètres entraînés	Coût mémoire	Usage privilégié
Ajustement complet	la totalité	très élevémodèle dédié, données massives	
Adaptation de faible rang	0,1 à 1 %	faible	le cas par défaut en entreprise
Adaptateurs discrets	0,01 à 0,1 %	très faible	terminaux, appareils embarqués
Apprentissage de consignes	aucun	nul	tâches ponctuelles

Tableau 2 – Les trois voies d'adaptation d'un modèle de langage

La règle est simple : si l'on peut énoncer des exemples de la tâche attendue, l'adaptation de faible rang suffit.

Génération augmentée par récupération

La génération augmentée ne demande jamais au modèle de connaître une information : elle lui fournit les passages pertinents au moment de la question. C'est le mécanisme central de l'usage d'entreprise.

- **Indexation :** les documents sont découpés en passages et projetés dans un espace vectoriel.
- **Recherche :** la question est projetée dans le même espace, et les passages proches sont récupérés.
- **Assemblage :** les passages filtrés par droits d'accès sont insérés dans la requête.
- **Génération :** le modèle répond en citant ses sources.
- **Refus :** si aucun passage ne soutient la question, le système répond qu'il ne sait pas.

> **Le refus est un comportement, pas un échec**
> Un système de génération augmentée qui répond systématiquement, même quand sa base ne contient pas la réponse, est plus dangereux qu'un système qui se tait. Concevoir le refus, le mesurer et l'afficher est une exigence de conception.

Agents conversationnels autonomes

Un agent conversationnel autonome ne se contente pas de répondre : il consulte des outils, planifie plusieurs étapes et vérifie son propre résultat. La différence avec un assistant tient à la boucle de décision et à l'instrument de vérification.

> **A retenir**
> - L'auto-attention remplace la récurrence et rend l'entraînement parallèle.
> - L'encodage positionnel est indispensable : l'attention seule est permutationnelle.
> - L'adaptation de faible rang est le compromis par défaut pour un usage d'entreprise.
> - La génération augmentée impose de concevoir le refus comme un comportement normal.

Chapitre 1.4

MLOps, éthique et responsabilité

Un modèle qui ne se déploie pas n'existe pas, et un modèle déployé sans audit est un risque.

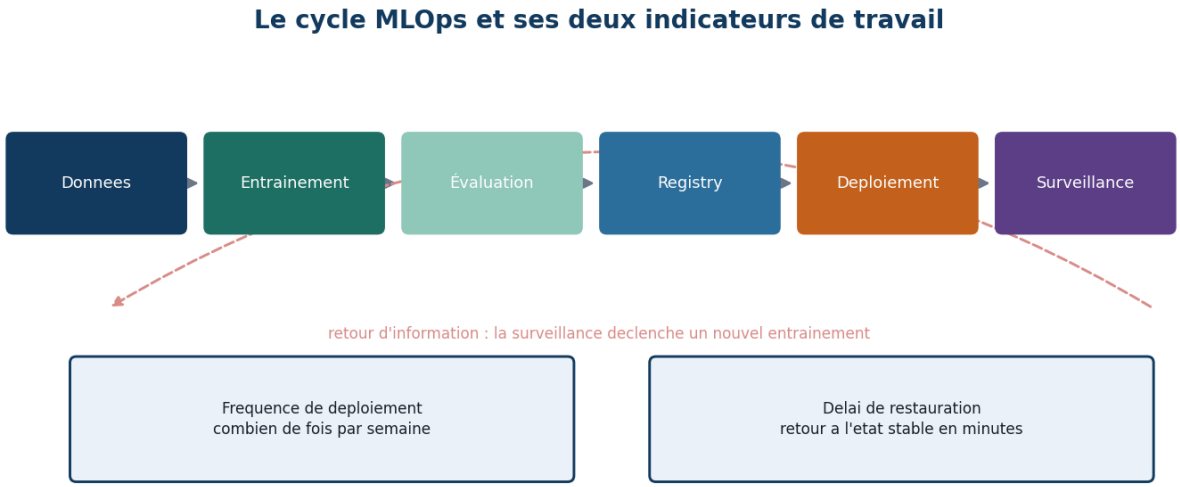


Figure 1 – Le cycle MLOps et ses deux indicateurs de travail

Le cycle d'industrialisation

- **Données :** collecte, contrôle qualité, historisation. Rien ne commence avant.
- **Entraînement :** paramétrage versionné, graine fixée, expérimentation suivie.
- **Évaluation :** sur jeux de référence figés, avec métriques métier et non seulement techniques.
- **Registre de modèles :** chaque version est enregistrée avec ses données d'entraînement et ses métriques. Sans cela, on ne peut pas rereconstruire un résultat.
- **Déploiement :** conteneurisation, infrastructure as code, déploiement reproductible.
- **Surveillance :** latence, erreur, dérive des données, qualité des requêtes entrantes.

Les deux seuls indicateurs qui comptent au début

Fréquence de déploiement : combien de fois par semaine l'équipe livre-t-elle une amélioration ? C'est le meilleur témoin de la maturité du processus.
Délai de restauration : combien de minutes pour revenir à un état stable après un incident ? C'est le meilleur témoin de la maturité de l'exploitation.
Tant que l'un des deux est mauvais, aucun autre investissement d'industrialisation n'est prioritaire.

```python
import hashlib
import json
import time
from dataclasses import dataclass, asdict
from pathlib import Path

import numpy as np


@dataclass(frozen=True)
class Signature:
"""Identite complete d'un modele : version, donnees, configuration.

Cette empreinte est ce qui permet, six mois plus tard, de rejouer
exactement le meme resultat. Elle doit etre calculee et stockee avec
le modele, jamais reconstruite a la main.
"""

version: str
empreinte_donnees: str
empreinte_code: str
configuration: dict
horodatage: float

def empreindre(self) -> str:
charge = json.dumps(asdict(self), sort_keys=True, default=str)
return hashlib.sha256(charge.encode("utf-8")).hexdigest()[:16]


def enregistrer(chemin: Path, sig: Signature, metriques: dict) -> str:
chemin.parent.mkdir(parents=True, exist_ok=True)
ident = sig.empreindre()
with chemin.open("a", encoding="utf-8") as f:
f.write(json.dumps({
"id": ident, "signature": asdict(sig), "metriques": metriques,
"train": time.time()}, default=str) + "\n")
return ident


def comparer(reference: dict, candidat: dict,
tolerance: float = 0.01) -> tuple[bool, dict]:
"""Verdict de promotion : on n'accepte que si rien ne se degrade.

Regle : la precision peut baisser dans la limite fixee, mais aucun
sous-groupe de population ne doit degrader au-dela du seuil. C'est la
seule condition qui evite d'échanger une regression globale
contre une regression invisible.
"""
ecarts = {}
accepte = True
for cle, attendu in reference.items():
if cle == "global":
continue
delta = candidat[cle] - attendu
ecarts[cle] = delta
if delta < -tolerance:
accepte = False
globale = candidat.get("global", 0.0) - reference.get("global", 0.0)
if globale < -tolerance:
accepte = False
return accepte, ecarts
```

Listing 1 — Registre de modèles et règle de promotion par sous-groupe

Conteneurisation et API d'inférence

- **Conteneurisation :** le modèle et ses dépendances voyagent ensemble. Un modèle n'est reproductible que si son environnement l'est aussi.
- **Version du format :** le format de sérialisation est lui-même versionné. Un fichier de poids sans sa version de format n'est pas un artefact, c'est une devinette.
- **API d'inférence :** contrat stable, limites de débit explicites, réponses d'erreur typées.
- **Repli :** si le service est indisponible, le système doit avoir un comportement défini — pas une page blanche.

Surveillance en production

Famille	Exemples	Ce qu'une anomalie signale
Technique	latence, débit, erreurs, mémoiredégradation d'infrastructure	
Qualité	longueur des réponses, refus, notes	dérive du modèle ou du service
Données	distribution des entrées, valeurs manquantes	dérive des données en amont
Affaires	taux de conversion, coût par requête	perte de valeur malgré une technique saine

Tableau 1 – Quatre familles de métriques de production

La famille « affaires » est celle qui déclenche les vraies décisions. Une latence dégradée se corrigé en quelques heures ; une valeur en baisse demande une investigation.

Éthique, biais et explicabilité

Trois exigences distinctes se confondent souvent. La **justice** interroge la répartition des erreurs entre groupes. L'**explicabilité** interroge la possibilité de justifier une décision. La **gouvernance** interroge la chaîne de responsabilité.

Exigence	Question posée	Moyen	Limite
Justice	les erreurs sont-elles réparties équitablement ?décomposition des métriques par groupe	corrige l'injustice, pas le modèle	
Explicabilité	peut-on justifier la décision ?	valeurs de substitution, attention, règles	post hoc : l'explication peut mentir
Traçabilité	sait-on quelle version a décidé ?	registre, journal d'appels	ne dit pas si c'était correct
Gouvernance	qui répond si c'est faux ?	règles, responsable désigné	institue sans résoudre

Tableau 2 – Les trois exigences, distinctes et cumulables

Le règlement européen sur l'intelligence artificielle (AI Act, 2024) impose, selon la classe de risque, une gestion documentée des risques, la qualité et la traçabilité des données, la transparence de l'usage, et un contrôle humain effectif pour les usages à haut risque. Le règlement sur la protection des données apporte, de son côté, minimisation, limitation des finalités et responsabilité. Un modèle déployé sans registre de versions et sans journal des décisions ne satisfait aucune des deux exigences.

Cadre juridique

Déployer un modèle sans seuil de refus. Un système qui répond toujours paraît plus disponible et se révèle plus dangereux : il produit une confiance que rien ne justifie. Le seuil de refus et la conduite à tenir quand il est franchi doivent être écrits avant le déploiement, pas improvisés à la première erreur coûteuse.

L'erreur la plus coûteuse en production

Audit de conformité en sept points

- **Finalité** documentée, connue des utilisateurs.

2. **Données** : provenance, qualité, base légale, durée de conservation.
3. **Métriques** décomposées par groupe pertinent.
4. **Registre** des versions, des données et des décisions.
5. **Seuil de refus** et conduite associée.
6. **Retour arrière** possible en moins d'une heure.
7. **Responsable** désigné, avec procédure d'escalade.

A retenir

- Fréquence de déploiement et délai de restauration résument la maturité MLOps.
- Un modèle n'est reproductible que si son environnement l'est aussi.
- Justice, explicabilité et gouvernance sont trois exigences distinctes et cumulables.
- Le seuil de refus doit être écrit avant le déploiement, jamais improvisé.

Abréviations, notation et ressources de consultation

Annexes

Annexes

Ce cours se termine par deux annexes de référence. La première donne la signification de chaque sigle employé, sous les deux formes — sigle et développé — car la litterature emploie indifféremment l'une ou l'autre. La seconde donne la signification de chaque symbole mathématique : elle permet de relire une démonstration sans revenir au chapitre dont elle est extraite, et de vérifier qu'un même symbole garde le même sens d'un chapitre à l'autre.

Chapitre A.1

Liste des abréviations

Les sigles ci-dessous sont ceux que le cours emploie. Chacun est donné avec sa forme développée et le sens précis qu'il prend ici, qui n'est pas toujours celui qu'on lui donne ailleurs.

Un sigle employé sans être développé oblige le lecteur à le reconnaître au passage, ou à l'abandonner. Le tableau ci-dessous ferme la liste de ceux que le cours emploie : il est exhaustif pour ce document, et non pour la discipline.

Sigle	Forme développée	Sens dans ce cours
ACP	Analyse en composantes principales	Projection sur les axes de plus grande variance, faite sans supervision.
AGI	Artificial General Intelligence	Intelligence artificielle générale, dont les capacités ne sont pas bornées à une tâche.
API	Application Programming Interface	Interface de programmation qu'un logiciel offre à un autre.
CNN	Convolutional Neural Network	Réseau convolutif, employé sur les images.
GELU	Gaussian Error Linear Unit	Fonction d'activation, qui approxime la distribution gaussienne de l'erreur.
LLM	Large Language Model	Grand modèle de langage, pré-entraîné sur un corpus massif.
LSTM	Long Short-Term Memory	Cellule récurrente à mémoire longue.
RLHF	Reinforcement Learning from Human Feedback	Alignement d'un modèle par renforcement sur des préférences humaines.
RNN	Recurrent Neural Network	Réseau récurrent.
SVM	Support Vector Machine	Machine à vecteurs de support.
t-SNE	t-distributed Stochastic Neighbor Embedding	Réduction de dimension non linéaire, employée pour la visualisation.

Tableau 1 – Abréviations employées dans le cours

Chapitre A.2

Table de notation

Les symboles ci-dessous gardent le même sens d'un chapitre à l'autre. Lorsqu'un chapitre doit réemployer un symbole pour un autre objet, il le redéclare sur la page même.

La notation est celle du cours : vecteurs en gras, matrices en majuscules grasses, scalaires en italique. Les espérances sont notées E, les lois de données D, les espaces de paramètres Θ.

Symbole	Signification
X	Matrice des observations, une ligne par exemple.
y	Vecteur des étiquettes associées.
ŷ	Prédiction produite par le modèle.
f_θ	Fonction apprise, paramétrée par θ.
θ	Vecteur de paramètres du modèle.
D	Loi de distribution des données.
d	Dimension de l'espace d'entrée.
C	Nombre de classes, pour un problème de classification.
N	Nombre d'exemples du jeu de données.
ℓ_i	Perte sur le i-ème exemple.
L	Perte moyenne sur le jeu entier.
□	Taux d'erreur.
σ(z)	Fonction logistique, qui borne une probabilité entre 0 et 1.
ReLU(z)	Fonction d'activation rectangulaire.
$\text{softmax}(z)_i$	Composante i de la distribution softmax du vecteur z.
p	Précision d'un détecteur.
r	Rappel d'un détecteur.
F_1	Moyenne harmonique de la précision et du rappel.

Tableau 1 – Symboles mathématiques et leur signification

> **Convention**
> Toute divergence constatée entre cette table et un chapitre signale une erreur de transcription, non une définition nouvelle : la table fait foi.

> **A retenir**
> - Un sigle employé sans être listé ici est un oubli de l'auteur, non une définition nouvelle.
> - Un symbole qui change de sens d'un chapitre à l'autre rend toute démonstration ininterprétable : c'est la principale cause des contresens que nous avons relevés.
> - Ces deux tables sont les seules parties du cours à maintenir à la main : le reste se relit tout seul.

Index des figures et des tableaux

Référence rapide du matériel visuel

Figures

1. Les quatre blocs de l'intelligence artificielle
2. Le cycle perception-décision-action de l'agent
1. Le cycle MLOps et ses deux indicateurs de travail

Tableaux

1. Trois stratégies d'exploration d'un espace d'états

2. Logique des prédicats et ontologies : deux usages distincts
1. Les algorithmes supervisés et ce qu'ils attendent des données
2. Perceptron, convolution, récurrence : trois hypothèses
1. Trois familles d'encodage positionnel
2. Les trois voies d'adaptation d'un modèle de langage
1. Quatre familles de métriques de production
2. Les trois exigences, distinctes et cumulables
1. Abréviations employées dans le cours
1. Symboles mathématiques et leur signification

Fin du document

Intelligence Artificielle Avancée & Federated Learning — Fondations mathématiques de l'apprentissage automatique, algorithmes fédérés, protection de la vie privée, défense contre les attaques, grands modèles de langage fédérés, systèmes embarqués, renforcement appris, applications critiques (santé, V2X), plateforme de référence FedX et feuille de route de recherche.
Ghiloufi Jamel, Architecte Logiciel Principal — Pixel Software Design
Département de Recherche & Développement — Recherche
Pixel Software Design — Édition 2026

© 2026 Pixel Software Design. Tous droits réservés.

Ce document est la propriété exclusive de Pixel Software Design. Toute reproduction, représentation, adaptation ou diffusion, totale ou partielle, par quelque procédé que ce soit et sur quelque support que ce soit, est interdite sans autorisation écrite préalable de l'auteur.

Document généré automatiquement le 29/09/2026 – mise en page Pixel Software Design.